AA1
Spring 2026

Machine Learning Competition

1. Introduction

The purpose of this homework is to solve a classification problem proposed as a competition in the ***Kaggle InClass*** platform, where each team of two members will try to get the maximum score. You can apply any of the concepts and techniques studied in class for exploratory data analysis, feature selection and classification.

2. Problem description

Context:

Channel State Information (CSI) in Wi-Fi refers to a detailed description of how a transmitted signal is affected by the wireless channel before reaching the receiver. Modern Wi-Fi systems use Orthogonal Frequency Division Multiplexing (OFDM), a multicarrier modulation scheme that splits the transmission into many closely spaced **subcarriers**, each carrying a portion of the data.

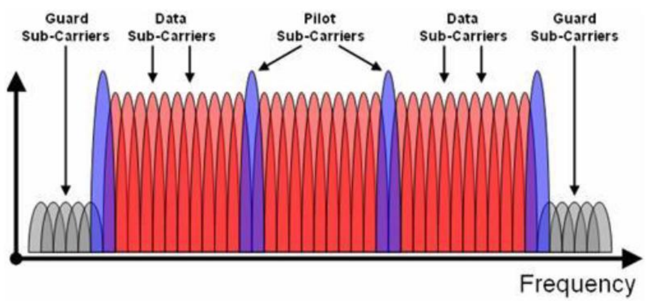

Fig. 1: OFDM signal in the frequency domain

As the signal travels through the environment, each subcarrier experiences different effects due to multipath propagation, reflections, shadowing, and fading. CSI captures these effects by reporting the amplitude and phase for every OFDM subcarrier, providing a fine-grained characterization of the channel. Earlier standards like IEEE 802.11n (aka Wi-Fi 4), 802.11ac (Wi-Fi 5), and 802.11ax (Wi-Fi 6) exposed CSI mainly for equalization, PHY rate adaptation and beamforming, but CSI is becoming increasingly central in the evolution of Wi-Fi standards, and this trend is explicitly reflected in the design goals of new amendments such as IEEE 802.11bf. In 802.11bf, CSI becomes one of the core building blocks of the Wi-Fi Sensing (WFS) framework. The key idea behind 11bf is that the physical layer's multipath signatures can be repurposed to infer information about the environment, such as motion, presence, occupancy, and localization. Recent studies have shown that Wi-Fi CSI can even be processed to extract fine-grained vital signs, including a person's heart rate[1], for example.

Machine learning is well suited for processing Wi-Fi CSI because modern Wi-Fi signals generate large volumes of high-dimensional data. A 20 MHz OFDM signal includes 64 subcarriers (see Fig. 1), and this number increases to 4096 for the 320 MHz channels supported in Wi-Fi 7. As a result, a single CSI sample may contain hundreds or even thousands of features. Analytical models struggle to handle such complexity, whereas ML techniques can learn these patterns directly and extract useful information.

[1] Sun J, Bian X, Li M. "Non-contact heart rate monitoring method based on Wi-Fi CSI signal". Sensors. 2024 March 26;24(7):2111. https://www.mdpi.com/1424-8220/24/7/2111

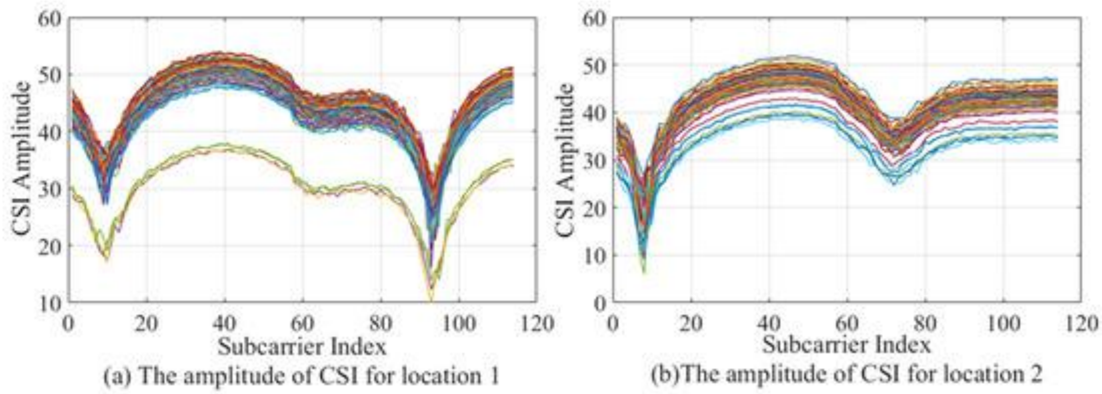

Fig. 2 Amplitude of different OFDM subcarriers measured in two different locations

Goal:

The goal of this exercise is to train a machine learning system capable of locating and tracking a Wi-Fi device by analyzing the CSI of its transmissions. The task is formulated as a classification problem, since the target device can occupy one of ten predefined positions (classes 0 to 9) relative to the tracking unit, as shown in the figure below:

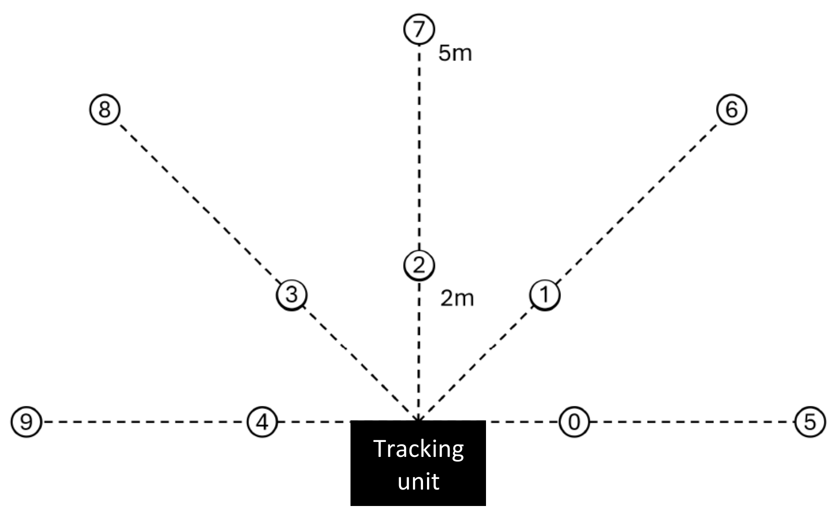

Fig. 3 possible positions of the target device, relative to the tracking unit (0 to 9)

The placements of the target device are at −90, −45, 0, 45, and 90 degrees with respect to the tracking unit. Two measurements are gathered from each of the aforementioned angles: one from 2 meters away and one from 5 meters away.

3. Dataset

The dataset for the classification problem is gathered by placing the tracking unit, which is a Wi-Fi device equipped with two antennas with 6 cm separation at a fixed location, and then passively measuring Wi-Fi traffic from a target device. As shown in Fig. 3, the target device is placed in 10 different locations around the tracking unit. For building the dataset, the target device periodically transmits ping packets (i.e. ICMP echo request packets). Transmissions from the target device are received at both of the tracking unit's antennas, and each antenna measures CSI data during the reception of each Wi-Fi frame preamble.

Each CSI sample contains the following features:

- `seq_ctrl`: uint16 sequence numbers.
- `aoa`: float32, synthetic feature generated in post-processing, representing the estimated angle of arrival (azimuth, in radians) computed from the phase difference between the two receiving antennas.
- `rssi1`: int8, received signal strength indicator measured by the first antenna.
- `rssi2`: int8, received signal strength indicator measured by the second antenna.
- Raw CSI data in the form of 64 complex numbers, first for antenna 1, then for antenna 2, structured as follows: `I0_1 Q0_1 I1_1 Q1_1 … I63_1 Q63_1 I0_2 Q0_2 … I63_2 Q63_2`
    - `In_X`: In-phase value of *n*th subcarrier ($0 \leq n \leq 63$) at antenna X ($X = 1$ or 2)
    - `Qn_X`: Quadrature value of *n*th subcarrier at antenna *X*
- `position` (label): int8, value between 0 and 9, corresponding to the position of the target device, as shown in Fig. 3

The complete dataset contains 16,111 samples. The classes are well balanced (there are more than ~1500 samples per class). The dataset is split into training (~80%) and test (~20%) subsets provided as two separate CSV files.

- The training set: `train.csv` contains 12,888 samples, each having 261 features and a label.
- The test set: `test_nolabels.csv` contains 3,223 samples following the same structure as the training set but not including the labels.

Note that all files contain a first numerical column used as the row ID (i.e. it is not a feature).

You will have to upload the predictions for the test set and Kaggle will compare your predictions with the ground-truth labels in order to compute a score.

4. The competition in Kaggle InClass platform

The competition is available at https://www.kaggle.com/t/f1f12dcb19d74c839ddfb87165ec1d75
Each group needs to create a Kaggle account to participate in the competition. Using this account, you will be able to download the dataset, participate in the competition forum, make submissions and see your scores on the live leaderboard.

One account per team: each team will use just one Kaggle account. For the registration use a single e-mail address with extension *@estudiantat.upc.edu*. The username must contain the last name of the group members, separated by underscore (name1_name2).

Code sharing: it is not allowed to share your code with other teams, but you can use the discussion Forum to post questions or comments.

Team alliances: alliances between groups are not allowed.

Submission limits: there is a limit of four submissions per day per team.

Competition timeline:
Start date: Mar 24th, 2026
End date: 11:59 PM, Jun 2nd, 2026

5. Evaluations

Metric:

The metric used by Kaggle to compute each submission score is the F1-score. However, in addition to the F1 score you can compute other performance metrics and include them in your final report.

Submission file format:

You should submit a CSV file with exactly 3,223 entries plus a header row. Your submission will show an error if you have extra columns or rows.

The file should have exactly 2 columns:

- **ID**: index corresponding to each sample from `test_nolabels.csv`.
- **position**: class prediction (0 to 9, corresponding to the predicted position).

Example:

```
ID,POSITION
0,0
1,5
2,1
3,0
```

Homework evaluation:

The evaluation of this assignment will be based on the quality of your report, feature engineering, justification of learning methods, optimization of hyperparameters, metrics evaluated, Python code and, rather than your position in the ranking, the value obtained for the target performance metric.

6. Code and report handling

Handling instructions:

Submit your work through Atenea. Save your code and report in a folder (name `AA1Comp_name1_name2`) and upload the compressed folder (zip, rar).

Code:

The code should contain all the Python scripts used for visualizing data, training and testing of models and creation of Kaggle submission files.

Report:

The report will be written using the IEEE two-column conference paper template provided through Atenea.

- The Abstract should contain a maximum of 200 words.
- The "Introduction" section should describe the problem
- The titles of the following sections should be:
  - "Feature analysis" (optional, if you use any method for feature selection, feature engineering or exploratory data analysis)
  - "Classification methods" (where you explain which of the models studied in class were used to solve the problem, including tricks to deal with class imbalance, use of ensemble classifiers: boosting, random forest, etc.)

    - “Evaluation metrics” (you can include other performance metrics in addition to the F1 score used in Kaggle, e.g. evaluation of P and R may explain the observed performance of F1)
    - “Experiments” (describe all the experiments, validation of hyperparemeters, etc).
    - “Results”
    - “Conclusions”
    - “References”
- You can use any number of subsections
- The maximum length of the report is 4 pages.
- All figures and tables should be one-column wide. Each figure/table should contain a numbered caption (below the figure/table). Figures and tables should be cited and explained in the text (ex. Fig.1, Fig2,…,Tab.1,…).
- All equations should be numbered and cited in the text.
- The document should contain useful numeric and graphical results and explanations, that is, those that give insight into the problem and proposed solutions. Explanations should be based on theoretical concepts studied in class.